

BAN404 Machine Learning - oversiktlig cheatsheet

Kopi av BAN404_cheatsheet, ryddet og oppdatert

2026-05-18

1. Fundamentals og bias-variance

Kjerneide. Vi observerer $Y = f(X) + e$ og estimerer f fra data.

- Parametrisk: antar form, f.eks. lineær modell. Få parametere, lav varians, men kan ha høy bias hvis formen er feil.
- Ikke-parametrisk: få formantakelser, f.eks. KNN, trær og splines. Fleksibelt, men kan få høy varians.
- Testfeil kan tenkes som: $MSE = \text{bias}^2 + \text{variance} + \text{irreducible error}$.
- For fleksible modeller: mer fleksibilitet gir ofte lavere treningsfeil, men testfeil kan øke pga. overfitting.
- Regularisering: høy α / λ gir enklere modell, mer bias og lavere varians.

Metrikker.

```
RSS = np.sum((y - pred)**2)
TSS = np.sum((y - y.mean())**2)
MSE = np.mean((y - pred)**2)
RMSE = np.sqrt(MSE)
MAD = np.mean(np.abs(y - pred)) # foreleser bruker ofte MAE/MAD
R2 = 1 - RSS / TSS
```

2. Data preparation og deskriptiv statistikk

Velg plott etter type variabler.

- Kontinuerlig Y, kontinuerlig X: scatterplot.
- Kontinuerlig Y, kategorisk X: boxplot.
- Kategorisk Y, kontinuerlig X: boxplot av X gruppert etter Y.
- Kategorisk Y, kategorisk X: krysstabell, gjerne normalisert.

Forelesers train/test-split.

```
np.random.seed(1234)
n = len(df)
ntrain = n // 2
ind = np.random.choice(np.arange(n), size=ntrain, replace=False)
mask = np.ones(n, dtype=bool)
mask[ind] = False
```

```
train = df.iloc[ind]
test = df.iloc[mask]
```

Ikke bruk `~ind` når `ind` er heltallsindekser. `~` fungerer her bare på boolske masker.

Dummies og manglende verdier.

```
X = pd.get_dummies(X_raw, drop_first=True)
X = X.astype(float).fillna(0)
y = (df["Direction"] == "Up").astype(int)
```

3. OLS - lineær regresjon

Modell. $y = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p + e$. OLS minimerer RSS.

- `statsmodels` gir p-verdier, standardfeil og konfidensintervall.
- `sklearn` er praktisk for prediksjon og pipelines, men gir ikke samme inferenstabell.
- Konfidensintervall gjelder forventet respons $E[Y|X]$; prediksjonsintervall gjelder ny observasjon og er bredere.

Forelesers statsmodels-mønster.

```
import statsmodels.api as sm
from ISLP.models import ModelSpec as MS, summarize
```

```
design = MS(["horsepower"]).fit(Auto)
X = design.transform(Auto)
y = Auto["mpg"]
res = sm.OLS(y, X).fit()
summarize(res)
```

```
X_new = design.transform(pd.DataFrame({"horsepower": [98]}))
res.predict(X_new)
res.get_prediction(X_new).conf_int(alpha=0.05) # CI
res.get_prediction(X_new).conf_int(obs=True, alpha=0.05) # PI
```

Uten ModelSpec.

```
Xc = sm.add_constant(X_train)
ols = sm.OLS(y_train, Xc).fit()
pred = ols.predict(sm.add_constant(X_test))
```

4. Ridge regression

Objektiv. Minimer $RSS + \alpha * \sum(\beta_j^2)$.

- L2-straff krymper koeffisienter mot null, men vanligvis ikke eksakt null.
- $\alpha = 0$ gir OLS.
- Stor α : sterk krymping, lavere varians, høyere bias.
- I eksamen 2025 brukte foreleser en egen `g()`-funksjon: y og X demeanes, men standardiseres ikke.

Analytisk ridge med demeaning.

```
b0 = y.mean()
yd = y - b0
Xd = X - X.mean(axis=0)

def ridge_coef(alpha, Xd, yd):
    p = Xd.shape[1]
    A = Xd.T @ Xd + alpha * np.eye(p)
    return np.linalg.solve(A, Xd.T @ yd)
```

```
b1 = ridge_coef(1000, Xd, yd)
pred = b0 + Xd @ b1
```

LOO for ridge, samme logikk som eksamen 2025.

```
def loo_ridge(alpha, X, y):
    n = len(y)
    b0 = y.mean()
    yd = y - b0
    Xd = X - X.mean(axis=0)
    ydpred = np.zeros(n)
    for i in range(n):
        idx = np.arange(n) != i
        b1 = ridge_coef(alpha, Xd[idx], yd[idx])
        ydpred[i] = Xd[i] @ b1
    return np.mean((yd - ydpred)**2)
```

Sklearn.

```
import sklearn.linear_model as skl
```

```

alphas = np.logspace(-4, 4, 200)
ridge_cv = skl.RidgeCV(alphas=alphas)
ridge_cv.fit(Xtrain, ytrain)

ridgemin = skl.Ridge(alpha=ridge_cv.alpha_)
ridgemin.fit(Xtrain, ytrain)
pred = ridgemin.predict(Xtest)

```

5. LASSO regression

Objektiv. Minimer $RSS + \alpha * \sum(\text{abs}(\beta_j))$.

- L1-straff kan sette koeffisienter eksakt lik null: variabelseleksjon.
- Bruk LASSO når mange prediktorer finnes, men bare noen forventes å være viktige.
- Standardisering er ofte viktig når variabler har ulike skalaer.

```

alphas = np.logspace(-4, 4, 200)
lasso_cv = skl.LassoCV(alphas=alphas)
lasso_cv.fit(Xtrain, ytrain)

lasso = skl.Lasso(alpha=lasso_cv.alpha_)
lasso.fit(Xtrain, ytrain)
pred = lasso.predict(Xtest)

```

Med StandardScaler.

```

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler(with_mean=True, with_std=True)
X_tr_s = scaler.fit_transform(X_train) # fit kun på train
X_te_s = scaler.transform(X_test)

lasso_cv.fit(X_tr_s, y_train)

```

6. Bootstrap

Kjerneide. Trekk n observasjoner med tilbakelegging, beregn statistikken, gjenta B ganger. De B verdiene approximerer samplingsfordelingen.

- Bruk `replace=True`.
- 95 prosent KI med persentilmetoden: 2.5 og 97.5 prosentil.
- For stikkprøvevarians: bruk `ddof=1`.

```

np.random.seed(42)
n = len(y)
B = 10000

boot_vars = np.array([
    np.var(y[np.random.choice(n, n, replace=True)], ddof=1)
    for _ in range(B)
])

ci = np.percentile(boot_vars, [2.5, 97.5])
plt.hist(boot_vars, bins=40)
plt.axvline(np.var(y, ddof=1), color="black", linestyle="--")

```

7. Logistisk regresjon

Modell. $P(Y=1|x) = 1 / (1 + \exp(-(\beta_0 + \beta_1 x)))$.

- Brukes når responsen er binær.
- Estimeres med maximum likelihood, ikke OLS.
- Koeffisienter er lineære i log-odds: $\log(p/(1-p))$.
- Standard terskel er 0.5, men ved ubalanserte klasser kan terskelen justeres på treningsdata.

Forelesers mønster med statsmodels.

```

import statsmodels.api as sm

X_train = sm.add_constant(train[["Lag2"]])
y_train = (train["Direction"] == "Up").astype(int)
m = sm.GLM(y_train, X_train, family=sm.families.Binomial()).fit()

X_test = sm.add_constant(test[["Lag2"]])

```

```

prob = m.predict(X_test)
pred = (prob > 0.5).astype(int)

```

```

y_test = (test["Direction"] == "Up").astype(int)
pd.crosstab(y_test.values, pred, rownames=["Actual"],
            colnames=["Predicted"])
np.mean(y_test == pred)

```

Formelvarianten foreleser også bruker.

```

import statsmodels.formula.api as smf
Weekly["y"] = (Weekly["Direction"] == "Up").astype(int)
m1 = smf.logit("y ~ Volume + Lag1 + Lag2 + Lag3 + Lag4 + Lag5",
               data=Weekly).fit()

```

8. KNN

Kjerneide.

- Regresjon: prediksjon er gjennomsnittet av y for de K nærmeste naboene.
- Klassifikasjon: sannsynlighet er andel naboer i klasse 1.
- Lav K : fleksibel, høy varians, kan overfitte.
- Høy K : glattere, høy bias, kan underfitte.
- Standardiser prediktorer når flere variabler inngår.

KNN fra scratch, slik foreleser bruker i GAM/øvinger.

```

def knn(x0, x, y, K=3):
    d = np.abs(x - x0)
    idx = np.argsort(d)[:K]
    return np.mean(y[idx])

```

LOO for valg av K .

```

best_K, best_err = None, np.inf
for K in range(1, 20):
    err = []
    for i in range(n):
        idx = np.arange(n) != i
        yhat = knn(x[i], x[idx], y[idx], K)
        err.append((y[i] - yhat)**2)
    if np.mean(err) < best_err:
        best_K, best_err = K, np.mean(err)

```

Sklearn.

```

from sklearn.neighbors import KNeighborsClassifier, KNeighborsRegressor

knn_clf = KNeighborsClassifier(n_neighbors=5)
knn_clf.fit(X_train, y_train)
pred = knn_clf.predict(X_test)
prob = knn_clf.predict_proba(X_test)[:, 1]

```

9. LDA, QDA og Naive Bayes

Når brukes hva?

- LDA: antar lik kovarians i alle klasser. Stabil ved lite data.
- QDA: egen kovarians per klasse. Mer fleksibel, trenger mer data.
- Naive Bayes: antar prediktorer er uavhengige gitt klasse. Nyttig med mange prediktorer.

Alle modeller estimerer posterior sannsynlighet $P(Y=k|X)$.

```

from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
↳ as LDA
from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis
↳ as QDA
from sklearn.naive_bayes import GaussianNB

for model in [LDA(), QDA(), GaussianNB()]:
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    print(type(model).__name__, np.mean(pred == y_test))

```

10. Polynomregresjon og trappefunksjoner

Polynom. Grad K gir kolonner 1, x , x^2 , ..., x^K . Velg grad med validation set eller CV.

```
K = 7
Xpoly = np.vander(age, K + 1, increasing=True)
m = sm.OLS(wage, Xpoly).fit()
```

Velg grad med validation set.

```
mse = []
for k in range(1, 11):
    Xtr = np.vander(age_train, k + 1, increasing=True)
    fit = sm.OLS(wage_train, Xtr).fit()
    Xte = np.vander(age_test, k + 1, increasing=True)
    mse.append(np.mean((wage_test - fit.predict(Xte))**2))
best_k = np.argmin(mse) + 1
```

Trappfunksjon.

```
train = train.assign(age_cut=pd.cut(train["age"], 4))
Xtr = pd.get_dummies(train["age_cut"], drop_first=True).astype(float)
Xtr = sm.add_constant(Xtr)
```

11. Regression splines

Kubisk spline med K knutepunkter.

Basis: 1, x , x^2 , x^3 , $h(x, x_i)$, ..., $h(x, x_K)$, der $h(x, x_i) = \max((x - x_i)^3, 0)$.

- Kubisk spline med K interne knutepunkter har $K + 4$ basisfunksjoner.
- Natural spline er lineær i halene.
- Velg knutepunkter med LOO eller faglig begrunnelse.

```
def h(x, xi):
    return np.maximum((x - xi)**3, 0)
```

```
X_sp = np.column_stack([
    np.ones(n), x, x**2, x**3,
    h(x, 3.0), h(x, 6.0)
])
```

```
m = sm.OLS(y, X_sp).fit()
```

ISLP BSpline.

```
from ISLP.transforms import BSpline
```

```
bs = BSpline(internal_knots=[3.5, 5.5], degree=3, intercept=True)
Xbs = bs.fit_transform(x.reshape(-1, 1))
m = sm.OLS(y, Xbs).fit()
```

12. GAM - Generalized Additive Models

Modell. $y = b_0 + f_1(x_1) + f_2(x_2) + \dots + f_q(x_q) + e$.

- Additiv: hver variabel får sin egen funksjon, men ingen interaksjoner med mindre de legges inn.
- Foreleser bruker backfitting med KNN-smoother for intuitisjon.
- Partiell residual for fj: $y - b_0 - \sum(f_k \text{ for } k \neq j)$.
- Demean residualene i hvert steg.

Backfitting-skjelett.

```
b0 = y.mean()
yd = y - b0
K = 20

f1 = b0 + np.array([knn(x0, x1, yd, K) for x0 in x1])
res = y - f1
res = res - res.mean()
f2 = b0 + np.array([knn(x0, x2, res, K) for x0 in x2])

for i in range(10):
    res = y - f2
    res = res - res.mean()
    f1 = b0 + np.array([knn(x0, x1, res, K) for x0 in x1])
```

```
res = y - f1
res = res - res.mean()
f2 = b0 + np.array([knn(x0, x2, res, K) for x0 in x2])
```

Eksamen 2025-logikk. Identifiser ikke-linearitet med scatter og/eller ΔR_2 mellom lineær og kvadratisk modell. Bygg så en GAM-lignende designmatrise med splines for de ikke-lineære variablene og lineære kolonner for resten.

```
for i, col in enumerate(var_names):
    xi = X[:, i]
    r2_lin = sm.OLS(y, sm.add_constant(xi)).fit().rsquared
    Xq = sm.add_constant(np.column_stack([xi, xi**2]))
    r2_quad = sm.OLS(y, Xq).fit().rsquared
    print(col, round(r2_quad - r2_lin, 3))
```

13. Decision trees

Kjerneide.

- Rekursiv binær splitting: finn variabel og splittpunkt som gir lavest feil.
- Regresjon: prediksjon i blad er gjennomsnittet av y .
- Klassifikasjon: prediksjon er majoritetsklasse; Gini brukes ofte som split-kriterium.
- Ubegrensede trær overfitter. Begrens med `max_depth`, `min_samples_leaf`, eller pruning.

Manuell første split.

```
from scipy.optimize import minimize_scalar
```

```
def RSS_split(s, x, y):
    R1 = y[x < s]
    R2 = y[x >= s]
    if len(R1) == 0 or len(R2) == 0:
        return np.inf
    return np.sum((R1 - R1.mean())**2) + np.sum((R2 - R2.mean())**2)
```

```
res = minimize_scalar(RSS_split, bounds=(x.min(), x.max()),
    args=(x, y), method="bounded")
```

Sklearn og pruning.

```
from sklearn.tree import DecisionTreeRegressor, DecisionTreeClassifier,
    plot_tree
from sklearn.model_selection import GridSearchCV
```

```
tree = DecisionTreeRegressor(min_samples_leaf=15, max_depth=3,
    random_state=0)
tree.fit(X_train, y_train)
plot_tree(tree, feature_names=X_train.columns, filled=False,
    fontsize=8)
```

```
path = tree.cost_complexity_pruning_path(X_train, y_train)
grid = GridSearchCV(DecisionTreeRegressor(random_state=0),
    {"ccp_alpha": path.ccp_alphas}, cv=5)
grid.fit(X_train, y_train)
best_tree = grid.best_estimator_
```

14. Bagging, random forest, boosting og BART

Forskjeller.

- Bagging: bootstrap mange trær og snitt prediksjoner. I sklearn: `RandomForestRegressor(max_features=p)`.
- Random forest: som bagging, men bare m tilfeldige prediktorer vurderes ved hvert split.
- Boosting: trær trenes sekvensielt; hvert nytt tre prøver å rette residualene.
- BART: Bayesiansk sum av trær, finnes i ISLP.bart.

```
from sklearn.ensemble import RandomForestRegressor,
    GradientBoostingRegressor
```

```
bag = RandomForestRegressor(n_estimators=500,
    max_features=X_train.shape[1],
    random_state=123)
```

```
bag.fit(X_train, y_train)
```

```
rf = RandomForestRegressor(n_estimators=500, max_features=6,
```

```

        random_state=123)
rf.fit(X_train, y_train)
pd.Series(rf.feature_importances_,
          index=X_train.columns).sort_values().plot.barh()

boost = GradientBoostingRegressor(n_estimators=1000,
                                  learning_rate=0.01,
                                  max_depth=2,
                                  random_state=123)

boost.fit(X_train, y_train)

```

Klassifikasjon.

```

from sklearn.ensemble import RandomForestClassifier

rfc = RandomForestClassifier(n_estimators=500, max_features="sqrt",
                           random_state=123)

rfc.fit(X_train, y_train)
prob = rfc.predict_proba(X_test)[: , 1]
pred = (prob > 0.5).astype(int)

```

15. Neural networks

Pensum. Foreleser har presisert at dette handler om MLPRegressor/MLPClassifier fra sklearn, ikke PyTorch, CNN eller RNN.

- hidden_layer_sizes=(15,): ett skjult lag med 15 noder.
- hidden_layer_sizes=(15, 8): to skjulte lag.
- ReLU: $\max(0, z)$.
- Optimeringen er ikke-konveks; bruk random_state.
- Standardiser X, særlig for neural networks.

```

from sklearn.neural_network import MLPRegressor, MLPClassifier
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

nn = MLPRegressor(hidden_layer_sizes=(15,), activation="relu",
                  max_iter=100000, random_state=0)
nn.fit(X_train_s, y_train)
pred = nn.predict(X_test_s)

dnn = MLPRegressor(hidden_layer_sizes=(15, 8), activation="relu",
                  max_iter=100000, random_state=0)

```

16. SVM

Kjerneide.

- Maximal margin classifier: finn hyperplan med størst margin.
- Support vector classifier: tillater feilklassifiseringer.
- I sklearn er C invers regulering: liten C = sterkere regulering, bredere/mykere margin, mer bias; stor C = svakere regulering, mer fleksibel, mer varians.
- Kernel SVM: polynom eller RBF for ikke-lineære grenser.
- RBF gamma: liten = glatt grense; stor = mer lokal/kompleks grense.

```

from sklearn.svm import SVC
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import confusion_matrix

param_lin = {"C": [0.01, 0.1, 1, 10, 100, 1000]}
tune_lin = GridSearchCV(SVC(kernel="linear"), param_lin,
                       cv=5, scoring="accuracy")
tune_lin.fit(X, y)

param_poly = {"C": [0.01, 1, 100, 1000], "degree": [1, 2, 3, 4]}
tune_poly = GridSearchCV(SVC(kernel="poly"), param_poly, cv=5)
tune_poly.fit(X, y)

param_rbf = {"C": [0.01, 1, 100, 1000], "gamma": [0.5, 1, 2, 4]}
tune_rbf = GridSearchCV(SVC(kernel="rbf"), param_rbf, cv=5)
tune_rbf.fit(X, y)

pred = tune_rbf.best_estimator_.predict(X_test)
confusion_matrix(y_test, pred)

```

17. Unsupervised learning: PCA og clustering

Unsupervised. Vi observerer bare X, ikke y. Brukes til eksplorativ/deskriptiv analyse og hypotesegenerering.

PCA

- PCA lager nye variabler Z1, Z2, ... som lineære kombinasjoner av X.
- Første principal component maksimerer varians.
- Andre component maksimerer gjenværende varians og er ukorrelert med første.
- Standardiser først når variabler har ulik skala. Foreleser bruker StandardScaler på USArrests.
- Loadings = vektorer for variablene. Scores = observasjonenes verdier på PC-ene.

```

from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

```

```

def summary_pca(fit):
    return pd.DataFrame({
        "Standard deviation": np.sqrt(fit.explained_variance_),
        "Proportion of Variance": fit.explained_variance_ratio_,
        "Cumulative Proportion":
            np.cumsum(fit.explained_variance_ratio_)
    })

scaler = StandardScaler(with_mean=True, with_std=True)
X_scaled = scaler.fit_transform(USArrests)
pca = PCA()
pca.fit(X_scaled)
summary_pca(pca)

loadings = pd.DataFrame(
    pca.components_.T,
    index=USArrests.columns,
    columns=[f"PC{i+1}" for i in range(pca.n_components_)])
scores = pca.fit_transform(X_scaled)

```

K-means og hierarkisk clustering

- K-means krever valgt K; minimerer within-cluster variation.
- Algoritme: start med sentre/tilordning, beregn sentroider, tilordne til nærmeste sentroid, gjenta.
- Hierarkisk clustering trenger ikke forhåndsvalgt K; gir dendrogram.
- Linkage: complete bruker maks avstand, single bruker min, average bruker snitt, ward minimerer varians.
- Standardiser før avstandsbaserte metoder.

```

from sklearn.cluster import KMeans, AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, cut_tree
from ISLP.cluster import compute_linkage

```

```

X_s = StandardScaler().fit_transform(X)

km = KMeans(n_clusters=3, random_state=1, n_init=10)
km.fit(X_s)
pd.crosstab(km.labels_, y_true)

hc = AgglomerativeClustering(distance_threshold=0, n_clusters=None,
                             linkage="complete")
hc.fit(X_s)
lm = compute_linkage(hc)
dendrogram(lm, no_labels=True)
clusters = cut_tree(lm, n_clusters=3).reshape(-1)

```

18. Quick reference

Standard imports.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.api as sm
import statsmodels.formula.api as smf
import sklearn.linear_model as skl

from ISLP import load_data
from ISLP.models import ModelSpec as MS, summarize
from ISLP.transforms import BSpline
from ISLP.cluster import compute_linkage
from ISLP.bart import BART

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeRegressor, DecisionTreeClassifier,
    plot_tree
from sklearn.ensemble import RandomForestRegressor,
    RandomForestClassifier
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.neural_network import MLPRegressor, MLPClassifier
from sklearn.svm import SVC
from sklearn.cluster import KMeans, AgglomerativeClustering
```

Confusion matrix.

```
cm = pd.crosstab(y_true.values, pred,
                 rownames=["Actual"], colnames=["Predicted"])
accuracy = np.mean(y_true == pred)
```

Hvis rader er faktisk klasse og kolonner er predikert klasse:

	Pred 0	Pred 1
Actual 0	TN	FP
Actual 1	FN	TP

Sensitivity/recall for klasse 1: $TP / (TP + FN)$. Specificity for klasse 0: $TN / (TN + FP)$.

Eksamenstaktikk fra tidligere eksamener.

- Bruk treningsdata/CV for modellvalg. Testsett brukes til endelig evaluering.
- Hvis flere valg er rimelige, forklar valget kort.
- Ved ubalanserte klasser: ikke stol blindt på accuracy; vurder terskel og klassespesifikke andeler.
- I deskriptive oppgaver: velg plott etter variabeltype og kommenter konkrete mønstre.
- I metodeoppgaver: forklar hva koden gjør, hvilken metode det er, og hvorfor datasettet demeanes/standardiseres.
- Vær pragmatisk: en tydelig og korrekt modell som svarer på spørsmålet er bedre enn en perfekt modell du ikke rekker.